

ACTIVIDAD 4

DESARROLLO DE APLICACIONES
MÓVILES



Alumno

ZAID CASTILLO HERMOSILLO
CARLOS SEBASTIAN CORONADO
ALTAMIRANO
LEONARDO PEREDA CHANEZ
CANEK LUGO SOTO

Profesor

ADALBERTO EMMANUEL
ROJAS PEREA

Actividad en clase

TaskMaster: gestor de tareas con Room

Modelado de datos

- Define una entidad Task con campos: id (autogenerado), title (String), description (String), isCompleted (Boolean) y dueDate (Long).

```
15 @Entity(tableName = "tasks")
16 data class Task(
17     @PrimaryKey(autoGenerate = true)
18     val id: Long = 0,
19     val title: String,
20     val description: String = "",
21     val isCompleted: Boolean = false,
22     val dueDate: Long = System.currentTimeMillis()
23 )
```

- Crea un DAO con métodos para insertar, eliminar, actualizar y listar tareas (usar Flow para reactividad).

```
@Dao
interface TaskDao {
    //Inserta una nueva tarea o reemplaza una existente si hay conflicto de clave primaria.//
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTask(task: Task): Long

    //Actualiza los datos de una tarea existente.//
    @Update
    suspend fun updateTask(task: Task)

    //Elimina una tarea de la base de datos.//
    @Delete
    suspend fun deleteTask(task: Task)

    //Obtiene la lista observable de todas las tareas ordenadas por estado (pendientes primero) y por fecha límite ascendente.//
    @Query("SELECT * FROM tasks ORDER BY isCompleted ASC, dueDate ASC")
    fun getAllTasks(): Flow<List<Task>>

    //Obtiene una tarea específica por su ID.//
    @Query("SELECT * FROM tasks WHERE id = :id")
    fun getTaskById(id: Long): Flow<Task?>
}
```

Configuración de la base de datos

- Implementa una clase AppDatabase abstracta con patrón Singleton.

```
@Database(
    entities = [Task::class],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun taskDao(): TaskDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        /**
         * Retorna la instancia única de AppDatabase (Singleton).
         * Si no existe, se sincroniza para crear la base de datos de manera segura.
         */
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "task_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
            }
        }
    }
}
```

- Define un repositorio (TaskRepository) que use el DAO para operaciones.

```
@Singleton
class TaskRepository @Inject constructor(
    private val taskDao: TaskDao
) {

    /**
     * Flujo reactivo con la lista completa de tareas.
     */
    val allTasks: Flow<List<Task>> = taskDao.getAllTasks()

    /**
     * Inserta una nueva tarea.
     */
    suspend fun addTask(task: Task): Long {
        return taskDao.insertTask(task)
    }

    /**
     * Actualiza una tarea existente.
     */
    suspend fun updateTask(task: Task) {
        taskDao.updateTask(task)
    }

    /**
     * Elimina una tarea.
     */
    suspend fun deleteTask(task: Task) {
        taskDao.deleteTask(task)
    }

    /**
     * Cambia el estado de completado de una tarea.
     */
    suspend fun setTaskCompleted(task: Task, isCompleted: Boolean) {
        taskDao.updateTask(task.copy(isCompleted = isCompleted))
    }
}
```

Integración con ViewModel

- Crea un TaskViewModel que exponga los datos mediante StateFlow y maneje eventos como addTask o deleteTask. Usa viewModelScope para operaciones asíncronas.

```
class TaskViewModel @Inject constructor(
    private val repository: TaskRepository
) : ViewModel() {

    /**
     * Estado observable de tareas expuesto como StateFlow reactivo.
     * Cualquier inserción, actualización o eliminación en Room actualizará automáticamente este flujo.
     */
    val tasks: StateFlow<List<Task>> = repository.allTasks
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )

    private val _uiEvent = MutableSharedFlow<TaskUiEvent>()
    val uiEvent: SharedFlow<TaskUiEvent> = _uiEvent.asSharedFlow()

    // Estado del diálogo de creación/edición
    private val _editingTask = MutableStateFlow<Task?>(null)
    val editingTask: StateFlow<Task?> = _editingTask.asStateFlow()

    private val _isDialogOpen = MutableStateFlow(false)
    val isDialogOpen: StateFlow<Boolean> = _isDialogOpen.asStateFlow()
}
```

Diseño de UI

- Implementa una pantalla con:
 - LazyColumn para mostrar tareas.

```
LazyColumn(  
    modifier = Modifier.fillMaxSize(),  
    contentPadding = PaddingValues(start = 16.dp, end = 16.dp, top = 8.dp, bottom = 88.dp),  
    verticalArrangement = Arrangement.spacedBy(12.dp)  
) {  
    items(  
        items = filteredTasks,  
        key = { it.id },  
        contentType = { "task" }  
    ) { task ->  
        TaskItem(  
            task = task,  
            onToggleCompleted = { isCompleted ->  
                viewModel.toggleTaskCompleted(task, isCompleted)  
            },  
            onEdit = {  
                viewModel.openEditDialog(task)  
            },  
            onDelete = {  
                viewModel.deleteTask(task)  
            }  
        )  
    }  
}
```

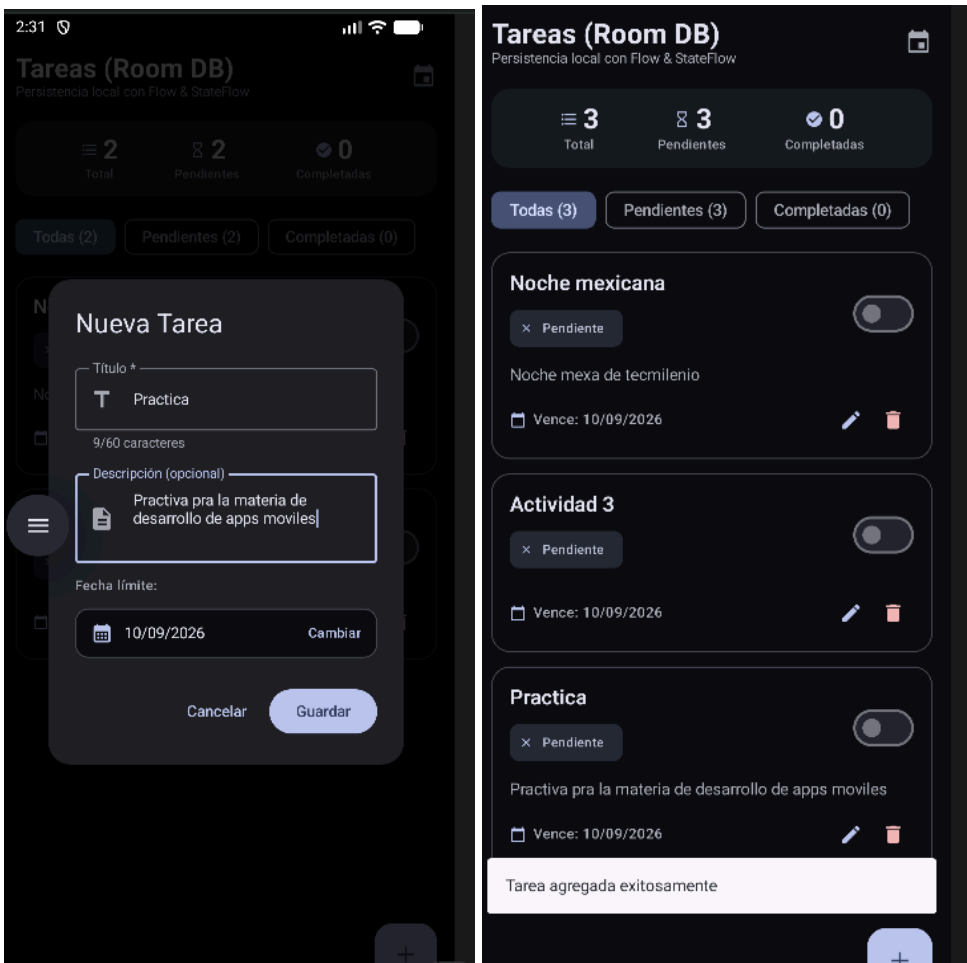
- FloatingActionButton para abrir un diálogo de creación.

```
FloatingActionButton = {  
    FloatingActionButton(  
        onClick = { viewModel.openCreateDialog() },  
        containerColor = MaterialTheme.colorScheme.primary,  
        contentColor = MaterialTheme.colorScheme.onPrimary  
    ) {  
        Icon(  
            imageVector = Icons.Filled.Add,  
            contentDescription = "Crear nueva tarea"  
        )  
    }  
}
```

- Switch para marcar tareas como completadas.

```
// Switch para marcar como completada  
Switch(  
    checked = task.isCompleted,  
    onCheckedChange = onToggleCompleted,  
    thumbContent = if (task.isCompleted) {  
        {  
            Icon(  
                imageVector = Icons.Filled.Check,  
                contentDescription = "Completada",  
                modifier = Modifier.size(SwitchDefaults.IconSize)  
            )  
        }  
    } else null,  
    colors = SwitchDefaults.colors(  
        checkedThumbColor = MaterialTheme.colorScheme.primary,  
        checkedTrackColor = MaterialTheme.colorScheme.primaryContainer  
    )  
)
```

Vista previa de la app. Se cambió a un tema oscuro con colores azules y rojos con baja saturación.

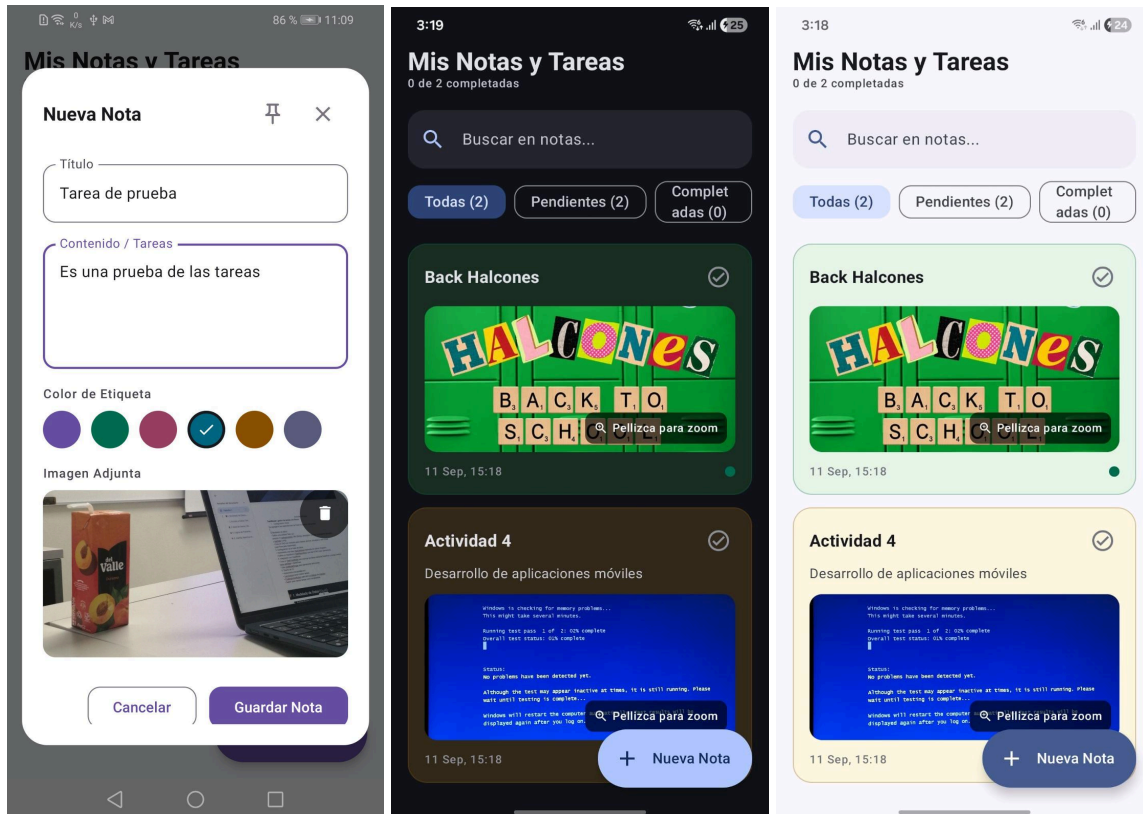


Base de datos visible en consola.

Live updates					
	id	title	description	isCompleted	dueDate
1	1	Noche mexicana	Noche mexa de tecmilenio	0	1789072158770
2	2	Actividad 3		0	1789072238353
3	3	Practica	Practiva pra la materia de desarrollo de apps moviles	0	1789072247522

Actividad 4

Pantallas de la aplicación con modo claro y oscuro.



Base de datos visible en consola.

id	title	content	date	isCompleted	imageUri	colorIndex	isPinned
5	Actividad 4	Terminar la actividad 4	2026-09-11T11:04:47.3	0	NULL	2	0
6	Certificado	Terminar el certificado	2026-09-11T11:05:17.0	0	NULL	4	0
7	Proyecto Final	Hacer el proyecto final	2026-09-11T11:06:01.1	0	NULL	5	0

Arquitectura técnica:

La aplicación está organizada bajo el patrón MVVM (Modelo - Vista - ViewModel). Esta estructura permite separar la aplicación en tres partes clave: la interfaz visual con la que interactúa el usuario, la lógica que controla y prepara la información en pantalla, y la base de datos que mantiene la persistencia de datos.

Gracias a esto, la aplicación es más ordenada, fácil de mantener y evita que la pantalla se congele al realizar operaciones de guardado o consulta.

Capa de Datos (Data Layer)

Es la encargada de almacenar y gestionar toda la información de manera local dentro del dispositivo:

- Modelo / Entidad (**Note.kt**): Representa la estructura de una nota, incluyendo su título, contenido, fecha de creación, estado (pendiente o completada), imagen adjunta, color asignado y si está fijada en la parte superior.
- DAO (**NoteDao.kt**): Objeto de acceso a datos que define las operaciones de base de datos (crear, consultar, actualizar y eliminar notas), emitiendo actualizaciones automáticas cada vez que la información cambia.
- Base de Datos (**NoteDatabase.kt**): Base de datos local SQLite administrada con Room, que incluye convertidores (Converters.kt) para guardar fechas y tipos de datos complejos de forma transparente.
- Repositorio (**NoteRepository.kt**): Funciona como un intermediario y fuente única de la verdad. Conecta la base de datos con el resto de la aplicación, ofreciendo una forma limpia y centralizada de solicitar o modificar notas.

Capa de Interfaz de Usuario (UI Layer)

Construida de forma moderna y declarativa con Jetpack Compose, esta capa se encarga de mostrar los datos al usuario y responder a sus acciones de forma fluida:

- Pantalla Principal (NotesScreen.kt): Es la vista central donde se muestran las notas, la barra de búsqueda en tiempo real y los filtros para alternar entre notas pendientes y completadas. Se actualiza automáticamente cada vez que ocurre algún cambio.
- Tarjetas de Notas Interactivas (NoteCard.kt):
 - Presentan la información de cada nota con colores personalizables.
 - Ofrecen feedback visual animado: al marcar una tarea como completada se muestra una animación de rebote y el texto se tacha.
 - Incorporan el gesto de deslizar para eliminar (Swipe-to-Dismiss), mostrando una advertencia visual y permitiendo deshacer la acción si fue por error.
- Visor de Imágenes con Gestos (ImagePinchZoomDialog.kt): Permite abrir imágenes adjuntas a pantalla completa para interactuar con gestos táctiles naturales: pellizcar para hacer zoom, arrastrar con el dedo para desplazarse y doble toque para ampliar o restablecer la vista.
- Creación y Edición (NoteEditDialog.kt): Ventana modal intuitiva donde el usuario puede redactar títulos, contenido, seleccionar colores o adjuntar imágenes.
- Diseño Adaptable a Cualquier Pantalla: La interfaz se acomoda automáticamente al tamaño y orientación del dispositivo, organizándose en una lista de una sola columna en teléfonos convencionales y distribuyéndose en cuadrículas de varias columnas en pantallas más amplias o tablets.

Decisiones de diseño

El diseño de la aplicación se centró en ofrecer una experiencia visual limpia, moderna y agradable. Para lograr que la interacción fuera natural y cómoda, se implementaron animaciones fluidas en cada acción: las notas aparecen y desaparecen con transiciones suaves, el tachado de tareas completadas responde con un efecto dinámico de rebote, y los colores cambian de forma armónica.

Además, se priorizó el uso de gestos táctiles intuitivos (como deslizar para borrar o pellizcar para hacer zoom en fotos) y una paleta de colores equilibrada con soporte para modo claro y oscuro, asegurando que el usuario reciba una respuesta visual inmediata a cada toque y disfrute de una navegación intuitiva y sin distracciones.

Uso de IA

Se utilizó el agente de IA Gemini para que pudiese analizar el código y poder escribir un análisis de la arquitectura, igualmente se utilizó durante el desarrollo para generar el código y que se asemejara al diseño que se creó en Figma.

Prototipo de diseño con flujos de interacción:

<https://www.figma.com/community/file/1680349159927511569/actividad4-desarrollomovil>